Day - 3: Operating Modes & Registers

▼ ARM Operating Modes | User | System | FIQ | IRQ | Abort | Undef

1. Overview of ARM Operating Modes

The ARM processor operates in a total of **8 different modes**: 7 standard operating modes plus 1 special mode known as Thumb mode (7 + 1 modes).

2. ARM Mode vs. Thumb Mode

The processor primarily switches between two instruction modes:

- **ARM Instruction Mode:** All instructions are executed in a **32-bit format**.
- **Thumb Instruction Mode:** The processor operates using **16-bit instructions**, which are transparently decomposed into full 32-bit ARM instructions during compilation without any loss in performance. About 50% to 60% of standard ARM instructions can be converted into Thumb mode.
- **Configuration:** Switching between these modes is controlled by the **T bit** in the **CPSR** (Current Program Status Register). Setting the `T bit = 0` configures the processor to ARM mode, while setting the `T bit = 1` configures it to Thumb mode.

3. Advantages of Thumb Mode

Thumb mode is incredibly beneficial for embedded systems with tight memory constraints:

- **Reduced Code Density:** Program size is reduced to about **65%**, saving roughly 35% of memory space compared to running entirely in ARM mode.
- **Increased Performance:** Because the program size is smaller, the processor's performance can increase by **up to 160%** compared to standard ARM execution.

- **Full 32-bit Hardware Access:** Even though Thumb mode uses 16-bit instructions, it still accesses standard **32-bit architecture**, meaning developers can still utilize 32-bit registers, the 32-bit ALU, 32-bit memory transfers, and the 32-bit shifter.

4. Key Architectural Differences

Feature	ARM Mode	Thumb Mode
Instruction Size	**32-bit**.	**16-bit**.
Instruction Count	Features **58** total instructions.	Features **30** total instructions.
Execution Style	Majority of instructions are **conditional**.	Only **branch instructions** are conditional.
Data Processing	Can access the barrel shifter and ALU simultaneously.	Has separate barrel shifter and arithmetic instructions.
CPSR Access	Allows read and write access to the status register (CPSR).	**No access** to read or write the CPSR.
General Registers	Uses 15 general-purpose registers (plus PC and CPSR).	Uses only **8 general-purpose registers** (plus Stack Pointer, Link Register, and Program Counter).

5. The 7 Standard Operating Modes

1. **User Mode:** An **unprivileged mode** that is open for general users to develop standard applications and programming.
2. **System Mode:** A **privileged user mode** that utilizes registers R0 to R15. It is specifically used for developing programs related to an Operating System (OS) or Real-Time Operating System (RTOS).
3. **Undefined Mode:** Used to handle errors such as **undefined instructions, variables, or functions**.
4. **Abort Mode:** A mode specifically designed to handle and manage **memory violation** situations.
5. **IRQ (Interrupt Request) Mode:** Used to generate and write Interrupt Service Routines (ISR) for interrupts with a **moderate or normal priority**.

6. **FIQ (Fast Interrupt Request) Mode:** Designed to handle **high-priority** interrupts. It operates faster by utilizing 8 dedicated registers (R8 to R14 plus 1 SPSR register).
7. **Supervisory Mode:** A privileged mode that is automatically entered in two specific situations: when the processor undergoes a **hardware reset**, or when it executes a **Software Interrupt (SWI)** instruction. It uses 3 specific registers (R13, R14, and 1 SPSR) and runs small programs known as "exceptional handlers".

▼ ARM Cortex Registers | PRIMASK | FAULTMASK | xPSR | BASEPRI

1. Introduction to ARM Cortex Special Registers

- **Location:** These special registers reside inside the ARM Cortex-M3 processor core itself.
- **Purpose:** Unlike general-purpose registers (R0 to R12), these special registers manage core configurations, interrupt masking, processor modes, and system status.

2. Interrupt Masking Registers

These registers are primarily used to manage and filter hardware interrupts and exceptions.

- **PRIMASK (Priority Mask Register):**
  - A **1-bit** register.
  - When set (value = 1), it disables almost all of the 255 potential interrupts, enabling **only Non-Maskable Interrupts (NMI) and Hard Fault exceptions** (which are high-priority).
- **FAULTMASK (Fault Mask Register):**
  - A **1-bit** register, similar to PRIMASK but slightly stricter.
  - When set, it masks all interrupts and hard faults, enabling **only Non-Maskable Interrupts (NMI)**.
- **BASEPRI (Base Priority Register):**

- An **8-bit** register used to assign a specific masking priority level threshold.
- **Rule:** In this system, a lower number signifies a higher priority.
- **How it works:** If you assign a value like `10` to BASEPRI, the processor will only allow interrupts with a higher priority (numbers `9` down to `1`). It will disable/mask any interrupt with priority `10` or above.
- Setting this bit to `0` completely disables the priority masking.

3. CONTROL Register

A **2-bit** register used to define the processor's operating mode and the active stack pointer.

- **Bit 0 (Thread Mode Selection):**
  - `0` = Privilege Thread Mode.
  - `1` = User Thread Mode.
- **Bit 1 (Stack Pointer Selection):**
  - `0` = Main Stack Pointer (MSP) (This is the default).
  - `1` = Process Stack Pointer (PSP).

4. xPSR (Program Status Register)

The **xPSR** is a comprehensive **32-bit** register that provides the current status information of the processor. It is conceptually subdivided into three distinct registers:

- **APSR (Application Program Status Register):**
  - Provides standard arithmetic logic flags.
  - Includes **N** (Negative), **Z** (Zero), **C** (Carry), **V** (Overflow), and **Q** (Sticky/DSP).
- **IPSR (Interrupt Program Status Register):**
  - Provides **exception information**.
  - Tracks the status of hard faults, NMIs, and interrupts generated from external peripherals.

- **EPSR (Execution Program Status Register):**
  - Provides execution state information, such as whether the processor is currently running in **Thumb mode**.
  - Also contains bits for **IT** (If-Then) and **ICI** (Interruptible-Continuable Instructions).

5. Software View & Other Core Registers

When debugging in an IDE, you will also see these standard core registers alongside the special registers:

- **R0 - R12:** General-purpose registers.
- **R13 (SP):** Stack Pointer (can act as either MSP or PSP).
- **R14 (LR):** Link Register, which stores return addresses.
- **R15 (PC):** Program Counter, which tracks the current instruction address

▼ ARM Cortex Operating Modes | Thread | Handler Mode

1. Overview of Operating Modes

The ARM Cortex-M3 processor categorizes its operations into two primary functional modes: **Thread Mode** and **Handler Mode**. These divide into three distinct operating modes:

- **Privileged Thread Mode**
- **User Thread Mode (Unprivileged)**
- **Privileged Handler Mode**

2. Thread Mode vs. Handler Mode

- **Thread Mode:** This is the primary mode used whenever a user application or program is executing. Depending on the application's needs, Thread Mode can execute with either privileged or unprivileged access.
- **Handler Mode:** The processor automatically enters this mode when it needs to serve an interrupt, hardware fault, or an exception. **Handler Mode always operates with full privileged access**.

3. Privileged vs. Unprivileged Access

- **Privileged Access:** This grants the operating mode complete, unrestricted access to all processor memories, instructions, and peripherals.
- **Unprivileged (User) Access:** This mode applies restrictions, allowing only limited access to the processor's memories, instructions, and peripherals. It is used when an application does not require additional hardware support.

4. Execution Flow & Mode Switching

- **System Reset:** Upon resetting the microprocessor, it automatically begins executing in **Privileged Thread Mode**.
- **The Control Register:** To move the processor from Privileged Thread Mode into the restricted User Thread Mode, developers must configure a specific 2-bit register known as the **Control Register**.
- **Returning to Privileged Mode:** If a program is running in the unprivileged User Thread Mode, **it cannot switch directly back to Privileged Thread Mode**. To regain privileged access, the program must initiate a software exception to force the processor into Handler Mode. From Handler Mode, the processor can then safely return to Privileged Thread Mode.
- After the processor finishes executing an exception or interrupt in Handler Mode, it goes back to whatever Thread Mode (User or Privileged) it was in prior to the interruption.

5. Handler Mode Specifics

The ARM Cortex-M3 processor is designed to support a vast number of interrupts and exceptions in its Handler Mode.

- It can generate and manage a total of **255 different handler functions** (ranging from 0 to 255).
- These are specifically broken down into **16 exceptions** and **240 interrupts**.

▼ Load and Store Process in ARM | LDR | STR

1. Introduction to Load & Store Architecture

- **ARM Concept:** ARM stands for Advanced RISC Machine. Because it is a RISC processor, it is not allowed to operate directly on data while it resides in the main memory.
- **Why it's used:** Handling memory directly requires more CPU cycles and execution time. Since RISC processors are specifically designed to execute tasks in very little time, the Load and Store architecture was implemented to maximize speed.

2. The Core Principle

The Load and Store architecture strictly separates memory access from data processing. It follows a three-step sequence:

1. **Load:** Data is moved from the main memory into the CPU's local registers.
2. **Process:** The CPU performs the necessary mathematical or logical operations on the data entirely within its registers.
3. **Store:** After the operation is complete, the resulting data is stored back from the registers into the main memory.

3. Key Instructions & Notation

To handle this architecture, ARM utilizes two primary instructions:

- **LDR (Load Register):** Used to load data from a memory location to a register.
- **STR (Store Register):** Used to store data from a register to a memory location.
- **Square Brackets [ ]:** In ARM programming, whenever a register is enclosed in square brackets, it signifies that the register is acting as a pointer indicating a **memory location**.

4. Use Case 1: The LDR Instruction

Example Scenario: Loading data into register `R2` from a memory location pointed to by `R0`.

- **The Pointer:** Register `R0` holds a specific memory address (e.g., `4000500C`).
- **The Data:** The system goes to that memory location (`4000500C`) and finds the data stored there (e.g., `1234ABCD`).

- **The Execution:** This data (`1234ABCD`) is successfully fetched from the memory location and loaded into register `R2`.

5. Use Case 2: The STR Instruction

Example Scenario: Storing data from register `R5` into a memory location pointed to by `R3`.

- **The Data:** Register `R5` holds the actual data you want to save (e.g., `FED2468`).
- **The Pointer:** Register `R3` acts as a memory pointer holding the target destination address (e.g., `40005050`).
- **The Execution:** The data (`FED2468`) is taken from `R5` and written directly into the memory location pointed to by `R3` (`40005050`).